
Python 金融数据分析

工具篇：AI 辅助编程与项目版本管理

主讲人： 王伟冠, 吕文俊
院 系： 经济学院金融系

上海大学

2026 年 9 月 24 日



1. VS Code 与学习路线
2. 虚拟环境搭建
3. Git 本地版本管理
4. 从 GitHub 获取与同步课程
5. AI 辅助编程：以 Qoder 为例
6. 综合练习与后续应用



1. VS Code 与学习路线
2. 虚拟环境搭建
3. Git 本地版本管理
4. 从 GitHub 获取与同步课程
5. AI 辅助编程：以 Qoder 为例
6. 综合练习与后续应用

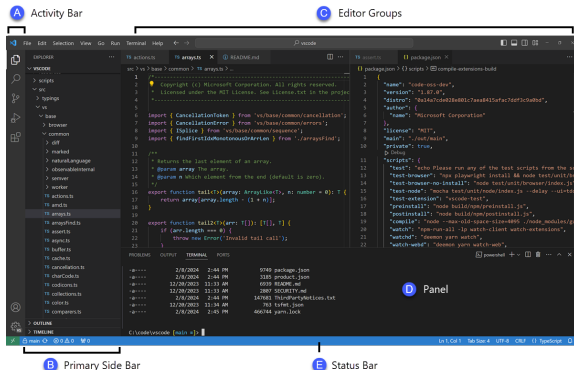


工具	在课程中的分工
VS Code	查看和编辑文件，运行脚本与 Notebook。
Python 与 uv	执行代码，管理项目的解释器与依赖。
Git 与 GitHub	记录版本，获取课程，保留练习并同步更新。
Qoder	根据任务读取项目，辅助编写、修改和验证代码。

贯穿全章的工作流程

认识编辑器 → 配置环境 → 学会 Git → 克隆课程。
再用 AI 辅助编程，检查修改与结果，保存可运行版本。

第一章讨论学习目标；本工具篇准备工作环境；后续章节学习 Python 与数据分析。



A 活动栏

切换文件、搜索、Git 等视图。

B 侧边栏

展开项目目录，找到代码文件。

C 编辑区

阅读和修改代码。

D 面板

打开终端，执行项目命令。

E 状态栏

查看分支等当前状态。

课堂操作

先找到“文件 → 打开文件夹”和“终端 → 新建终端”。
下一节创建 py4fi-course 后，打开该目录，观察环境文件。

图片：VS Code 官方界面指南；图中为官方示例项目，字母对应右侧中文说明。

1. VS Code 与学习路线
- 2. 虚拟环境搭建**
3. Git 本地版本管理
4. 从 GitHub 获取与同步课程
5. AI 辅助编程：以 Qoder 为例
6. 综合练习与后续应用



同一台电脑，两个金融分析项目

旧项目依赖 pandas 旧版接口，新项目需要新版功能。
如果共用一套包，升级新项目的依赖可能让旧代码无法运行。

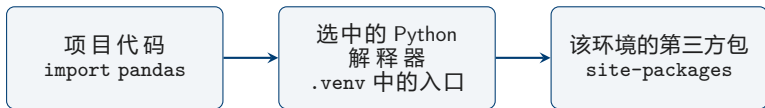
项目 A：历史回测

项目 A 的代码
项目 A 的 .venv
适合旧代码的 Python 与包版本

项目 B：新数据分析

项目 B 的代码
项目 B 的 .venv
适合新代码的 Python 与包版本

- **隔离依赖**：各项目分别安装包，减少版本冲突。
- **方便复现**：记录 Python 与依赖版本，在其他电脑上重建环境。
- **.venv** 是项目中的一个文件夹；代码和数据放在它的外面。



- 创建环境时，工具配置 Python 入口、环境配置和独立的包目录。
- 该入口关联一个基础 Python 安装；运行时识别环境位置，使用对应的包搜索路径，默认不包含系统的第三方包目录。
- `.venv/pyvenv.cfg` 记录基础 Python 等环境信息。
- 激活环境会把其命令目录放到当前终端的 PATH 前面，使输入 `python` 时优先找到它。

关键区别

虚拟环境隔离 Python 包；它不创建新的操作系统，也不是虚拟机。

机制参考：Python 官方 `venv` 文档

工具	主要用途
Python	解释并执行我们的代码。
venv	Python 自带的虚拟环境创建工具，通常搭配 pip 装包。
conda	管理环境与包，也能处理许多非 Python 依赖。
uv	管理 Python 版本、创建虚拟环境、安装包并记录项目依赖。

从零搭建的顺序

安装 uv → 新建项目 → 创建 .venv → 安装分析包 → 运行并检查 → 保存环境说明文件。

- 本次练习从个人目录中的**全新文件夹**开始，无需现成环境文件。
- Windows 使用 **PowerShell**；macOS 使用**终端 (zsh)**。
- 标注“两系统相同”的命令，在两种终端中都可直接运行。

Windows: 打开 PowerShell

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

macOS: 打开终端

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

安装完成后，关闭并重新打开终端；两系统相同：

```
uv --version
```

- 显示 uv 的版本号，说明终端已经能找到 uv 命令。
- 此时安装的是**环境管理工具**；分析用的包稍后再安装。
- 无需提前安装 Python；后续创建环境时，uv 可下载缺少的 Python。

命令来源：uv 官方安装文档

Windows / PowerShell

```
cd $HOME
mkdir py4fi-course
cd py4fi-course
```

macOS / 终端

```
cd ~
mkdir py4fi-course
cd py4fi-course
```

以下命令两系统相同：

```
uv init --bare --python 3.11
uv python pin 3.11
```

- `mkdir` 创建文件夹，`cd` 进入文件夹；后续命令都在这里执行。
- `uv init --bare` 创建最小的 `pyproject.toml`，初始依赖为空。
- `uv python pin` 把项目选用的 Python 版本写入 `.python-version`。
- 这一步是在写项目说明，**还没有安装分析包。**

Windows / macOS 相同：在 py4fi-course 文件夹中运行

```
uv venv --python 3.11 --prompt py4fi-course
```

- uv 找到或下载 Python 3.11，再创建当前项目的 .venv 文件夹。
- --prompt py4fi-course 设置激活后的显示名称；环境目录仍是 .venv，分析包稍后安装。

直接调用环境中的 Python，无需先激活：

Windows / PowerShell

```
.\.venv\Scripts\python.exe -c "import sys;  
print(sys.executable)"
```

macOS / 终端

```
./venv/bin/python -c "import sys; print(  
sys.executable)"
```

观察结果

输出路径应位于 py4fi-course 的 .venv 中。
Windows 的命令目录叫 Scripts，macOS 的叫 bin。

Windows / macOS 相同：

```
uv add numpy "pandas>=2.2,<3.0" matplotlib
uv add jupyterlab ipykernel
uv pip list
uv run python -c "import pandas as pd; print(pd.__version__)"
```

- 前三个包用于数值计算、表格分析和绘图；后两个支持 Notebook。
- pandas 限定为 2.x，以配合课程现有代码；引号让终端完整传入版本条件。
- uv add 把需求写入 pyproject.toml，解析兼容版本，生成或更新 uv.lock，并安装到 .venv。
- 包还会依赖其他包，因此 uv pip list 通常比手动添加的名单长。

怎样判断成功？

最后一行能导入 pandas 并显示版本号；上一页已确认解释器位置。

示意片段: 描述项目需要什么, 具体下限可能因安装时间不同而变化。

```
[project]
name = "py4fi-course"
version = "0.1.0"
requires-python = ">=3.11"
dependencies = [
    "numpy>=2.0",
    "pandas>=2.2,<3.0",
    "matplotlib>=3.9",
]
```

- name、version 是**项目**的名称和版本, 不是 Python 版本。
- requires-python 是允许的 Python 范围; .python-version 指定本项目选用的版本, 如 3.11。
- dependencies 是直接依赖; >= 为最低版本, < 为上限, == 为固定版本。
- 日常用 uv add 修改依赖; 这个文件本身不包含 Python 或包代码。

文件或目录	保存什么	如何管理
<code>pyproject.toml</code>	项目信息、Python 范围、直接依赖要求	随项目保存
<code>.python-version</code>	选用的 Python 版本，如 3.11	随项目保存
<code>uv.lock</code>	解析后的准确包版本及间接依赖；可含不同平台的条件	随项目保存，由 uv 更新
<code>.venv/</code>	本机实际使用的 Python 入口、配置和已安装包	本机生成，不跨电脑复制
<code>requirements.txt</code>	另一种常见依赖清单，可写版本约束；内容不一定完整锁定	本流程无需额外创建

从需求到安装结果

`pyproject.toml` (要求) → `uv.lock` (解析结果) → `.venv` (本机环境)。
环境文件用于**重建**环境；它们不等于环境本身。

参考：uv 项目结构与依赖管理

推荐方式：在项目文件夹内运行；Windows / macOS 相同

```
uv run jupyter lab
uv run python analysis.py
```

第一行启动 JupyterLab；第二行用于运行自己已保存的 `analysis.py`。
`uv run` 会检查项目环境并在需要时同步依赖，再执行命令，无需手动激活。

也可以手动激活，再直接输入 `python`：

Windows / PowerShell

```
.\.venv\Scripts\Activate.ps1
python --version
deactivate
```

macOS / 终端

```
source .venv/bin/activate
python --version
deactivate
```

- 激活只影响当前终端的命令查找；`deactivate` 恢复，不会删除环境。
- PowerShell 若限制激活脚本，直接使用上方的 `uv run` 即可。

- ① 用 VS Code 打开 py4fi-course 文件夹，安装 Microsoft 的 **Python** 和 **Jupyter** 扩展。
- ② 新建或打开 .ipynb 文件，点击右上角 **Select Kernel**，从 Python 环境中选择本项目的 .venv。

对应的解释器路径

Windows: py4fi-course\.venv\Scripts\python.exe

macOS: py4fi-course/.venv/bin/python

在 Notebook 单元格运行，两系统相同：

```
import sys
import pandas as pd
print(sys.executable)
print(pd.Series([0.01, -0.02, 0.03]).mean())
```

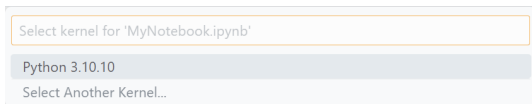
路径应指向项目的 .venv，平均收益率约为 0.00667。

若提示找不到包：先核对内核路径，再在项目终端运行 `uv add 包名`。

终端激活环境，并不自动替换 Notebook 已选的内核。

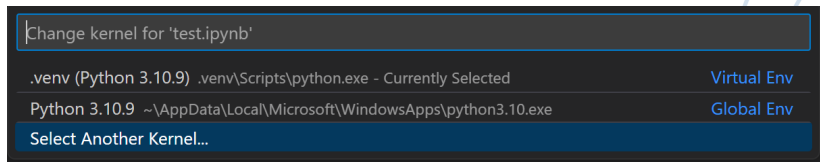
参考：VS Code 官方 Notebook 指南

1. 点击 Notebook 右上角的 Select Kernel, 打开内核选择器。



若列表只有最近用过的环境，选择 **Select Another Kernel...**，再查看 Python 环境。

2. 对照环境路径，找到当前项目的.venv。



本课程要核对什么？

图中 `.venv\Scripts\python.exe` 是虚拟环境入口；全局环境另列一行。
图示版本为 3.10；课堂应选择自己创建的 Python 3.11 环境，并运行 `sys.executable` 核对路径。

图片：VS Code 内核管理与 Notebook 指南；不同版本的菜单外观可能不同。

- ① 保存代码、数据说明及三个文件：pyproject.toml、uv.lock、.python-version。 **不复制.venv文件夹。**
- ② 在另一台电脑安装 uv，进入收到的项目文件夹。
- ③ 用保存的依赖记录重建，并检查运行结果。

Windows / macOS 相同：

```
uv sync --locked
uv run python -c "import sys; print(sys.executable)"
uv run python -c "import pandas as pd; print(pd.__version__)"
```

- uv sync --locked 按已有锁定文件同步环境；如果声明与锁文件不一致，会提示错误，需先由项目维护者更新锁文件。
- Windows 与 macOS 的环境目录结构和安装文件可能不同，应分别重建。
- 复现包版本是基础；分析结果还取决于数据、随机种子及运行平台等。

课堂回顾

现在已能创建和重建环境。接下来学习 Git，再从 GitHub 获取课程及其环境文件。

1. VS Code 与学习路线
2. 虚拟环境搭建
- 3. Git 本地版本管理**
4. 从 GitHub 获取与同步课程
5. AI 辅助编程：以 Qoder 为例
6. 综合练习与后续应用



为项目记录可以追溯的版本

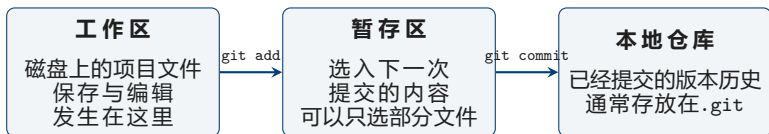
Git 是分布式版本控制工具。每次提交（commit）记录一个项目快照，并保存作者、时间、说明和版本之间的关系。

- **比较修改**：自己改了哪些文件、哪些行，计算逻辑是否改变？
- **查阅历史**：哪个版本能运行？某个公式为什么被修改？
- **尝试新方案**：在分支上实验，验证后再合并到主线。
- **恢复与协作**：找回已记录的内容，与他人交换已提交的版本。

本课程中的例子

为 `mean_demo.py` 记录“计算三个数的平均值”，再记录“增加一个输入数字”，无需保存多个“最终版”文件。

参考：Git 官方入门教程



- **保存文件**更新工作区；**add** 选取内容；**commit** 记录版本。
- 暂存的是执行 `add` 时的内容；随后又改了文件，需要再次 `add`。
- 提交在本地完成，无需联网；**push** 才把提交发送到远程仓库。
- Git 不会自动记录每次编辑；没有提交的内容不属于已保存的版本历史。

参考：git add git push

- ① 从 git-scm.com/install 按系统说明安装 Git，重新打开终端。
- ② 进入前面创建的 `py4fi-course` 文件夹，执行下列命令。

```
git --version
git init -b main
git config user.name "Your Name"
git config user.email "you@example.com"
git status
```

- `init` 创建本地仓库，`main` 是本练习的主分支名称。
- 把示例姓名和邮箱换成自己的；此处配置只作用于当前仓库。
- 姓名与邮箱用于标注提交作者，不是 GitHub 的登录信息。
- 若 `uv` 已创建 Git 仓库，跳过 `init`；用 `git branch --show-current` 查看当前分支名，后续按实际名称操作。

参考：Git 初始化与作者配置

在项目根目录创建或补充.gitignore，每行一个忽略规则：

```
.venv/  
__pycache__/  
.ipynb_checkpoints/  
.env  
data/raw/
```

- **提交**：代码、说明、.gitignore，以及前面生成的 pyproject.toml、uv.lock、.python-version。
- **忽略**：本机环境、缓存、存放密钥的.env 和本例的原始数据目录。
- 数据另行保存来源与获取说明；Notebook 提交前检查输出中的数据内容。
- .gitignore 只影响未跟踪文件，不会自动移除已经提交的文件或历史。

添加前先看一眼

使用 `git status` 查看文件清单，再按文件名添加；忽略文件仍保留在电脑上。

参考：Git 忽略规则

在 py4fi-course 中新建 mean_demo.py, 输入并保存:

```
values = [2, 4, 6]
mean = sum(values) / len(values)
print(mean)
```

先预测, 再运行

三个数的和是 12, 个数是 3, 平均值应为 $12/3 = 4$ 。

```
uv run python mean_demo.py
```

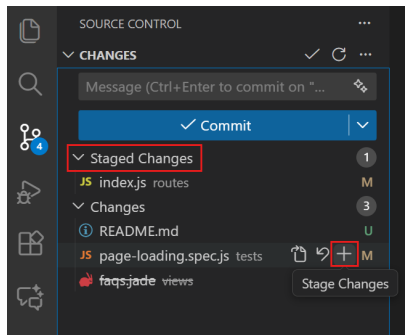
- 终端应输出 4.0; sum 求和, len 统计元素个数。
- 本节关注如何保存、运行和记录版本; Python 语法会在后续章节展开。
- 下一页把这份可运行程序与环境文件一起提交。

承接上一页的平均值程序：先保存文件，再在项目目录执行。

```
uv run python mean_demo.py
git status
git add mean_demo.py .gitignore
git add pyproject.toml uv.lock .python-version
git diff --cached
git commit -m "Add mean example"
git log -5 --format="%s"
```

- 核对平均值输出为 4.0，再记录这一版本。
- `diff --cached` 展示即将提交的内容；有遗漏时补充 `add`。
- `commit -m` 写明这次做了什么；最后一行查看最近 5 次提交的标题。
- 如果提示 `nothing to commit`，说明当前没有可提交的新内容。

参考：提交与查看历史



Changes: 尚未暂存

先点击文件查看差异，再点击该文件旁的+，选入下一次提交。

Staged Changes: 已经暂存

对应 `git diff --cached`；这里的内容将进入提交。

Message 与 Commit

写明本次完成的任务，核对暂存内容后提交。

图中的 M 表示已修改，U 表示尚未跟踪。

迁移到平均值练习

找到 `mean_demo.py`，核对修改并运行，再暂存该文件；
暂存后若继续编辑，需要再次暂存，才会把后续修改加入提交。

图片：VS Code 官方暂存与提交指南（仅展示左侧操作区域）；示例文件名不同，操作相同。

```
git status
git diff -- mean_demo.py
git diff --cached -- mean_demo.py
git log -5 --format="%s"
```

- status：区分未跟踪、未暂存、已暂存的文件。
- diff：比较工作区与暂存区；diff --cached：比较暂存区与上次提交。
- 新建且未跟踪的文件不会出现在普通 diff 中，应直接打开检查。
- VS Code 的**源代码管理**面板也可查看文件差异、暂存和提交。

只想撤销暂存，保留代码修改

已有首次提交后，执行 `git restore --staged mean_demo.py`。
不带 `--staged` 的 `restore` 会改写工作区文件，含义不同。

参考：git diff git restore

```

80 const samplePeople = [
105   },
106   {
107     name: "Amara Okafor",
108     position: "Marketing Director",
109     bio: "Creative strategist promoting sustainable
110   }
111 ];
112
113
114
115- res.render('index', { title: 'City Bikes - Urban B
116- });
117
118- /* GET news page. */
119- router.get('/news', function(req, res, next) {
120
121
122+   res.render('index', {
123+     title: 'City Bikes - Urban Bike Sharing'
124+   });
125+ });
126
127+ /* GET news page.
128+ router.get('/news', function(req, res, next) {
  
```

左侧：比较基准

红色区域表示删去或被替换的内容。

右侧：待检查版本

绿色区域表示增加或替换后的内容。

先看比较对象

从 Changes 打开时，看未暂存修改；从 Staged Changes 打开时，看将提交的修改。

对平均值程序提出三个问题

改动是否只涉及目标文件？平均值公式是否正确？修改后的输入是否已经运行验证？

图片：VS Code 官方差异编辑器示例；图中为 JavaScript，只用于认识差异视图，无需学习图中代码。

先提交当前版本，确认 `git status` 显示工作区干净。

```
git switch -c mean-update
```

把 `mean_demo.py` 的输入改为 `[2,4,6,8]`，预期平均值为 5。

```
uv run python mean_demo.py
git diff -- mean_demo.py
git add mean_demo.py
git commit -m "Add a fourth value"
git switch main
git merge mean-update
```

怎样理解分支？

新提交先记在 `mean-update`，合并后主线也包含这次修改。
普通分支共用工作目录；切换会更新文件，不会自动隔离未提交的编辑。

合并出现冲突时，理解两边意图，编辑为正确结果，再运行、暂存并提交。

参考：`git switch` 分支与合并

- **Git** 是版本管理工具；**GitHub / Gitee** 是托管远程仓库的平台。
- **commit** 保存到本地；**push** 上传提交；**pull** 获取并整合远程更新。
- 从他人已有仓库开始，用 `git clone` 仓库地址，无需再执行 `init`。

可选练习：在平台创建空仓库并完成认证后，回到本地主分支执行。

```
git remote add origin https://github.com/USER/REPO.git  
git push -u origin main
```

将 `USER/REPO` 替换为实际地址；`origin` 是远程仓库的简称。上面假设此前未配置 `origin`；后续可用 `git push` 同步新提交。

本节最低目标

完成一次 “修改 → 查看差异 → 运行验证 → 提交”。

下一节用 `clone` 获取老师已有的仓库；此处发布个人仓库的命令可课后尝试。

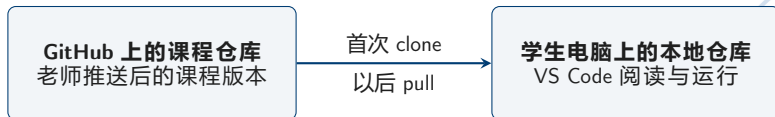
参考：`git push` 远程协作

1. VS Code 与学习路线
2. 虚拟环境搭建
3. Git 本地版本管理
- 4. 从 GitHub 获取与同步课程**
5. AI 辅助编程：以 Qoder 为例
6. 综合练习与后续应用



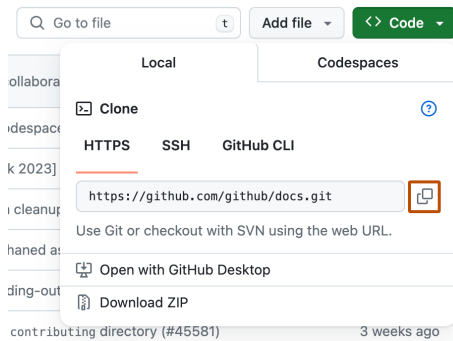
本课程仓库 (repo, 即 repository)

<https://github.com/aiqfin/python-for-finance>
aiqfin 是仓库所属组织，python-for-finance 是仓库名称。



- 课程仓库当前公开：浏览和通过 HTTPS 克隆通常不需要 GitHub 账号。
- **clone** 建立本地副本并保存远程地址；**pull** 获取更新并整合到当前分支。
- **Download ZIP** 只下载文件快照，解压目录不能直接用 Git pull 更新。
- 老师在本机保存或提交还不够；**推送到 GitHub 后**，学生才能拉取到更新。

仓库与默认分支于 2026-09-24 核对；参考：GitHub 克隆指南。



1. 打开课程仓库主页

先确认地址中的组织和仓库名。

2. 点击绿色 Code 按钮

选择 Local 下的 HTTPS。

3. 点击复制图标

把地址用于 Git clone 或 VS Code 的克隆入口。

图中是官方示例仓库

课堂请使用下方课程地址，不复制截图里的 github/docs。

本课程的克隆地址

<https://github.com/aiqfin/python-for-finance.git>

图片：GitHub 官方克隆指南。

已学会 Git 和 uv；在 py4fi-course 外的父目录打开终端。

```
git clone https://github.com/aiqfin/python-for-finance.git
cd python-for-finance
git remote -v
git branch --show-current
```

- clone 自动新建 python-for-finance 文件夹；首次执行即可。
- origin 应指向课程地址；当前默认分支是 master。
- 在 VS Code 中选择“文件 → 打开文件夹”，打开刚克隆的目录。

也可以直接用 VS Code 克隆

打开命令面板，选择 **Git: Clone**，粘贴课程 HTTPS 地址，选择保存位置并打开。

两种方式任选其一；从 GitHub 克隆后已经有仓库，**不用再次 git init**。

前面空目录练习使用 main；课程仓库使用 master，以实际分支名为准。

参考：VS Code 仓库与远程操作。

在 python-for-finance 根目录运行：

```
uv sync --locked
uv run --locked python -c "import sys; print(sys.executable)"
```

- 使用仓库已有的 `pyproject.toml` 和 `uv.lock` 建立本机 `.venv`。
- 已有项目不需要重新 `uv init` 或逐个添加课程依赖。
- 在 VS Code 打开对应章节的 `notebooks/`，选择仓库根目录的 `.venv` 内核。
- 先阅读根目录和本章 README，按要求获取数据，再运行 Notebook。

为什么某些文件没有下载下来？

Git 只传输已跟踪且已推送的文件。本仓库忽略生成的课件 PDF、环境目录及部分大型数据。

PDF 按各章说明编译或从老师提供的渠道获取；缺失数据按章节说明准备。

若锁文件与依赖声明不一致，保留报错并反馈；不要为了安装而随意改锁文件。

参考：`uv` 锁文件与环境同步；文件范围依据本仓库 `.gitignore`。

先保存编辑器内容，在仓库根目录检查：

```
git status  
git branch --show-current
```

确认工作区干净、位于 master，且没有自己的本地提交后：

```
git pull --ff-only origin master  
uv sync --locked  
git log -3 --format="%s"
```

- --ff-only 只允许分支直接前进；若历史分叉，会停止，避免自动生成合并。
- 环境同步成功后，重新打开有更新的 Notebook，核对老师公布的内容。
- Already up to date. 表示当前分支无需再整合该远程分支的更新。

已有练习修改怎么办？

先按下一页保存到个人分支，再更新课程主分支。Notebook 的输出变化也可能被 Git 识别为修改。

参考：Git pull 文档。

首次开始练习时，从课程 master 创建个人分支：

```
git switch -c my-work
```

完成一段练习，保存 Notebook，检查并提交指定文件：

```
git status
git add "Chapter2 - Python基础语法/notebooks/02_Python_Basics.ipynb"
git diff --cached
git commit -m "Complete chapter 2 practice"
```

- 按实际修改的文件替换路径；提交前按前文设置自己的作者姓名和邮箱。
- 已有 my-work 时用 `git switch my-work`，不要重复创建。
- 若已在 master 上编辑但尚未提交，先创建个人分支再提交；确认 Git 状态。

本地提交与上传作业是两回事

`commit` 只保存在自己电脑。克隆公开课程不授予向老师仓库 `push` 的权限；作业按老师要求提交。

前提：个人练习已提交，git status 显示工作区干净。

```
git switch master
git pull --ff-only origin master
git switch my-work
git merge master
```

- master 用于接收课程更新；my-work 保存练习，合并后同时包含两边的提交。
- 合并成功后执行 `uv sync --locked`，再运行本次涉及的例子。
- 同一位置两边都改过可能发生冲突；保留双方意图，处理后检查、运行、提交。

遇到报错时，先判断属于哪一类

提示本地修改将被覆盖：先保存并提交自己的修改。

提示无法快进：主分支已有分叉，先请老师协助检查。

Notebook 合并冲突：不熟悉其格式时先求助，不要用丢弃全部修改来解决。

Fork 是在自己 GitHub 账号下建立仓库副本，适合以后上传与协作；接收课程更新不要先 Fork。

- ① 展示本地仓库路径、origin 地址和当前分支名。
- ② 确认老师已推送一次更新，拉取后打开对应文件，指出新增内容。
- ③ 展示 Notebook 使用的解释器路径，说明它属于哪个项目。

讨论

为什么下载 ZIP 后不能直接 pull？为什么老师 commit 后学生可能仍看不到更新？

为什么运行 Notebook 后，下一次同步前仍需查看 git status？

- ① 展示本地仓库路径、origin 地址和当前分支名。
- ② 确认老师已推送一次更新，拉取后打开对应文件，指出新增内容。
- ③ 展示 Notebook 使用的解释器路径，说明它属于哪个项目。

讨论

为什么下载 ZIP 后不能直接 pull？为什么老师 commit 后学生可能仍看不到更新？

为什么运行 Notebook 后，下一次同步前仍需查看 git status？

参考答案（下一步展开）

ZIP 不包含完整 Git 仓库；老师还需要 push；Notebook 保存的输出也可能改变文件。

验收关注仓库地址、分支、具体文件更新及解释器路径，不能只看“命令已执行”。

1. VS Code 与学习路线
2. 虚拟环境搭建
3. Git 本地版本管理
4. 从 GitHub 获取与同步课程
- 5. AI 辅助编程：以 Qoder 为例**
6. 综合练习与后续应用



- 在 Qoder 中打开本地项目，说明需要完成的任务和涉及的文件。
- 让 AI 辅助阅读代码、保存修改、执行命令，并报告运行结果。
- 在 VS Code 中阅读代码、运行 Notebook，核对计算和输出。

本章统一使用 Qoder 示例

本章使用 Qoder CN 桌面端；同类工具可沿用相同的任务描述与验证流程。

一次任务的交付物

修改后的项目文件、实际运行结果，以及对修改内容的解释。

参考：Qoder CN 官方快速入门

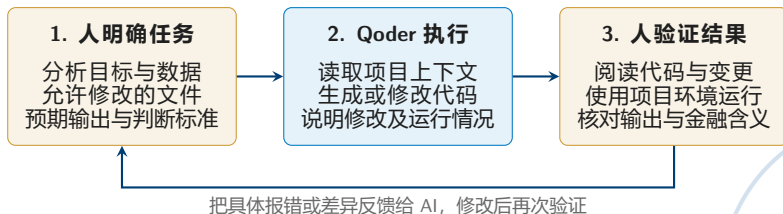
- ① 从 Qoder CN 官方下载页安装桌面端并登录。
- ② 选择**编程**，打开前面创建的 py4fi-course，继续平均值与收益率练习。
- ③ 选择**本地模式**，确认工作区完整路径与当前分支，再开始新任务。
- ④ 在 VS Code 中打开同一个目录，选好 .venv 解释器或 Notebook 内核。
- ⑤ 先发送一个阅读任务：“列出本项目的环境文件，解释它们的作用，暂不修改文件。”

两个窗口，一份项目文件

课程仓库 python-for-finance 用于同步教材；本节切回 py4fi-course。两个项目各有自己的环境和 Git 历史；核对 VS Code 与 Qoder 打开的完整路径。

桌面端入口名称可能随版本变化；以官方指南和当前界面为准。

参考：Qoder CN 官方快速入门



向 AI 交代环境和边界

使用本项目的 `.venv`；先说明修改方案；保留已有编辑；修改后列出变更文件、执行的命令，以及仍需我确认的结果。

- 交给 AI 修改前，先保存 VS Code 中的编辑；同一文件避免同时修改。
- AI 完成后查看磁盘上的最新内容；不要用未保存的旧内容覆盖新修改。
- 人负责核对数据、公式与经济解释，AI 的完成提示是验证的起点。

AI 编程是什么？

用自然语言描述需求，让 AI 辅助编写、解释和修改代码，再通过实际运行与结果检查，逐步完成任务。

组成	在课程项目中的作用
大语言模型	根据需求与已有信息，生成代码和解释。
项目上下文	提供当前代码、数据字段、报错和环境信息。
工具	在授权范围内读取文件、保存修改、执行命令。

- 编程智能体（Agent）把模型与工具结合，按任务需要反复执行和调整。
- 回复中的代码要经过运行；程序能运行，还要检查公式和数据口径。
- 学习时请 AI 解释关键步骤，并尝试自己修改输入、预测输出。

软件操作参考：Qoder CN 官方快速入门

问题：让 AI 算出平均值，并确认程序真的运行过

请读取已有的 `mean_demo.py`，把输入改回 `[2,4,6]`，运行并说明结果。

模型核对并修改的程序

```
values = [2, 4, 6]
mean = sum(values) / len(values)
print(mean)
```

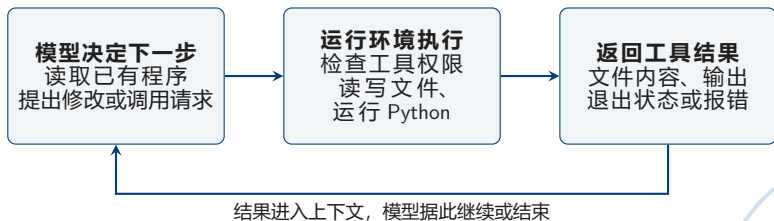
人先手算： $(2 + 4 + 6)/3 = 4$ 。

执行工具的调用示意

```
tool: run_command
arguments:
  command: >
    uv run python mean_demo.py
result:
  stdout: "4.0"
  exit_code: 0
```

- **Tool calling (工具调用)**：模型提出工具名称与参数，运行环境执行并返回结果。
- 文件要由写入工具保存；Python 解释器负责计算；模型再解释输出。
- 上面的接口名称是教学示意，**不是 Qoder 的实际 API**。执行记录才是运行证据。

概念参考：Qoder 任务执行原理。



沿着平均值例子观察执行记录

读取项目约定 → 保存程序 → 运行命令 → 得到 4.0 → 核对手算值。

- 上下文包括当前任务、读到的文件和工具返回值；模型不天然知道磁盘内容。
- 工具失败时，根据具体报错修正；缺少工具或权限时，应说明未完成的步骤。
- 退出状态为 0 说明程序正常结束；**是否算对，仍要核对输入和公式。**

概念参考：Qoder Agent 主循环与工具执行。

每次都要说明“用哪个环境、文件放哪里、怎样验证”，可以写入项目根目录的 AGENTS.md。

```
# 项目约定
```

- 用中文解释关键步骤。
- 使用项目 `.venv`；运行脚本用 `uv run python 文件名.py`。
- 输入数据放 `data/`，运行结果放 `outputs/`。
- 修改前查看 `Git` 状态，保留已有编辑。
- 修改后运行相关示例，列出命令与实际输出。
- 未运行的检查要明确说明。

- 它是供 Agent 阅读的 Markdown 文本，可用 VS Code 编辑，文件名采用大写 AGENTS.md。
- 项目共享约定可纳入 Git；只写简短、稳定、能执行和检查的要求。
- “今天只修改平均值程序”属于本次提示词；不必写成长期规则。

示例为本课程自编；文件机制参考：Qoder CLI 静态记忆文档。

- ① 在练习项目根目录保存 AGENTS.md，先检查里面的命令和目录是否真实可用。
- ② 开始任务时要求：“先读取项目根目录的 AGENTS.md，概括与本次任务相关的约定。”
- ③ 提交具体任务，例如修改 mean_demo.py；检查实际命令是否使用项目环境。
- ④ 约定变化时修改该文件，查看差异并随项目提交。

区分产品与作用范围

Qoder CLI 文档明确支持项目 AGENTS.md，可用 `/memory` 查看加载情况。桌面端课堂练习先显式要求读取；不要把 CLI 的自动加载规则直接套到所有客户端。

规则的作用与边界

文件为模型提供指令，**不保证每条要求都被正确执行**，也不等于系统权限。检查运行路径、输出和 diff，才能判断任务有没有按约定完成。

参考：Qoder CLI 的 AGENTS.md 与加载机制。

从重复任务出发

不同练习都需要“读代码、手算小样本、运行、核对输出”，可以整理为一个验算技能。

示例：技能目录 `verify-example` 中的 `SKILL.md`

```
---
name: verify-example
description: 验算简短 Python 教学示例时使用。
---
1. 阅读指定程序，说明输入、计算公式与输出。
2. 选一个可以手算的小例子，先写出预期值。
3. 用项目环境运行程序，记录命令和实际输出。
4. 对比预期值；若有差异，解释原因并重新验证。
5. 汇报修改的文件，以及尚未检查的内容。
```

文件头说明名称和适用任务，正文给出步骤；技能还可附带脚本、模板和参考资料。

格式参考：Agent Skills 规范；上例是教学用流程，并未安装到当前项目。

- ① 打开“扩展 → 技能”，先阅读技能用途、说明及附带脚本。
- ② 安装已有技能；或通过“添加技能”导入自己准备的 SKILL.md 或技能 ZIP。
- ③ 在任务输入框输入 /，从建议列表选择技能，再补充文件与验收要求。
- ④ 检查执行过程和结果；技能提供流程，实际读写与运行仍通过工具完成。

沿用平均值练习

选择 `verify-example` 技能后输入：

“验算 `mean_demo.py`。输入为 `[2,4,6]`，先手算再运行；只检查该文件。”

- Qoder 也可根据任务匹配已安装技能；初次学习先明确选择，便于观察。
- Skill 不会重新训练模型；导入技能后仍需核对依赖和权限是否满足任务要求。
- 把文件放进任意目录不代表已经安装；先确认“已安装”和建议列表中可见。

操作参考：Qoder CN 桌面端技能指南；入口随版本变化。

概念	回答的问题	在平均值任务中的例子
本次提示词	这次做什么？	为三个数字计算平均值。
AGENTS.md	项目怎么协作？	用项目环境，保留已有修改。
Skill	这类任务怎么做？	先手算，再运行，最后核对。
Tool calling	具体怎么执行？	保存文件、执行命令、取得输出。
Git	如何记录与比较？	查看程序差异，提交验证后的版本。

从平均值迁移到收益率

任务改为累计收益率后，项目约定与验算流程仍可复用；
需要重新确定金融公式与预期结果，不能直接沿用平均值的算法。

- ① 在练习目录写三条项目约定，说明 Python 环境、修改范围和验证方式。
- ② 让 Qoder 读取约定，把平均值输入改为 [3,6,9]，先手算再运行。
- ③ 在执行记录中分别找到一次读取、一次修改和一次运行，并说明返回了什么。
- ④ 按前述方式导入验算技能，再让它检查 [2,4,9]，解释输出。

讨论：这些信息应该放在哪里？

“一直用项目环境”；“这次输入为三个数字”；“先手算再运行”；“运行后的输出是 5.0”。

- ① 在练习目录写三条项目约定，说明 Python 环境、修改范围和验证方式。
- ② 让 Qoder 读取约定，把平均值输入改为 [3,6,9]，先手算再运行。
- ③ 在执行记录中分别找到一次读取、一次修改和一次运行，并说明返回了什么。
- ④ 按前述方式导入验算技能，再让它检查 [2,4,9]，解释输出。

讨论：这些信息应该放在哪里？

“一直用项目环境”；“这次输入为三个数字”；“先手算再运行”；“运行后的输出是 5.0”。

参考答案（下一步展开）

预期平均值分别为 6 和 5。四类信息分别对应：项目约定、本次提示词、技能流程、工具结果。

后面的收益率例子也要先确定预期值，再对照执行结果。

课堂提示词示例：在桌面端关联 py4fi-course 后输入

请新建 `returns_demo.py`，计算价格序列 `[100, 110, 99]` 的两期简单收益率和累计收益率。

仅使用 Python 标准库；用本项目的 `.venv` 运行；不修改其他文件。先解释公式，再编写和运行代码。

输出两期收益率和累计收益率，保留两位小数并显示百分号；列出运行命令，解释累计收益率为什么不等于两期收益率之和。

- **目标与输入**：指定计算任务与完整的示例数据。
- **约束**：说明文件范围、Python环境和可用依赖。
- **验收**：明确输出格式、运行证据和需要解释的问题。
- 若结果有误，反馈实际输出与预期差异；若报错，附完整报错。

在 VS Code 阅读 AI 生成的文件，并与下面的计算核对：

```
prices = [100, 110, 99]
r1 = prices[1] / prices[0] - 1
r2 = prices[2] / prices[1] - 1
total = (1 + r1) * (1 + r2) - 1
print(f"Period 1: {r1:.2%}")
print(f"Period 2: {r2:.2%}")
print(f"Total: {total:.2%}")
```

手算验收

两期收益率为 +10% 和 -10%；累计收益率为
 $(1 + 10\%)(1 - 10\%) - 1 = -1\%$ ，也等于 $99/100 - 1$ 。

- 在项目目录运行 `uv run python returns_demo.py`，核对三个输出。
- 两期简单收益率直接相加得到 0%，不代表持有期累计收益率。
- 将末期价格改成 100，先预测再运行：累计收益率应为 0%。

- ① 在代码审查区域查看 diff，逐项检查修改的文件与代码行。
- ② 结合 VS Code 或 Git 命令区分未暂存与已暂存内容，核对修改范围。
- ③ 对具体代码行提出反馈，再让 Qoder 修改并重新运行验证。
- ④ 确认公式、输出和文件范围后，选择需要提交的内容。

审查时先确认修改范围

结合 Git 状态检查自己的编辑和其他未提交内容，不把所有修改都归因于本次 AI 任务。

同一套 Git 概念

Qoder 面板、VS Code 源代码管理与终端命令，都围绕同一项目的版本状态工作。

参考：Qoder CN 官方快速入门

回到 py4fi-course, 先保存、验证并提交两期版本:

```
uv run python returns_demo.py
git add returns_demo.py
git diff --cached
git commit -m "Add two-period returns example"
git status
```

- 输入保持 [100,110,99], 核对累计收益率为 -1% 。
- 平均值程序、项目约定等若还有修改, 另行检查并提交相关文件。
- 工作区干净后, 从练习主线创建分支, 再让 AI 扩展收益率程序。

```
git switch -c agent-returns
```

复用已经学过的方法

前面手动修改平均值程序; 现在由 AI 辅助修改, 仍需查看差异、运行验证和记录版本。

在 agent-returns 分支中给 Qoder 的任务

先查看 Git 状态，保留已有修改。仅修改 `returns_demo.py`，使它支持任意长度的正价格序列（至少两个价格）。

用本项目的 `.venv` 运行，验证 `[100,110,99]` 的累计收益率为 -1% ，`[100,110,100]` 为 0% 。完成后解释 diff 和运行结果，暂不提交或推送。

学生审查并运行验证后，保存这次任务的版本：

```
git diff -- returns_demo.py
uv run python returns_demo.py
git add returns_demo.py
git diff --cached
git commit -m "Support multiple return periods"
```

也可以在审查后明确让 Agent 提交指定文件；一次提交对应一个清楚的任务。确认后用 `git switch main`、`git merge agent-returns` 合并回练习主线。

1. VS Code 与学习路线
2. 虚拟环境搭建
3. Git 本地版本管理
4. 从 GitHub 获取与同步课程
5. AI 辅助编程：以 Qoder 为例
- 6. 综合练习与后续应用**



- ① 沿用 `py4fi-course` 的环境与 Git 历史，整理两期收益率程序。
- ② 手算核对 `[100, 110, 99]`，找到或补充提交两期版本。
- ③ 在扩展分支中让 Qoder 支持至少两个正价格；已完成时直接检查该版本。
- ④ 核对三期示例 `[100, 110, 99, 108.9]`：累计收益率应为 8.9%。
- ⑤ 查看差异、运行验证，提交扩展版本，再合并回主分支。

提交作业时能说明

程序如何运行；两个版本有什么区别；用什么手算结果核对；哪些结论由自己判断。

后续 NumPy、pandas 和可视化练习沿用此流程，重点转向数据与分析方法。

Thanks for your attention!

Q & A

